

上海交通大学课程小组项目论文

Enterprise-RAG-Guard 企业知识库安全 RAG 平台

课程：文本分析与大模型

项目网站：enterprise-rag-guard-deploy.onrender.com

代码仓库：github.com/qiaoqiaoyining-source/enterprise-rag-guard-deploy

小组成员：黎鸿基、乔伊宁、孙钰婷、徐晨飞、刘婧祺

2026 年 6 月

摘要

随着大语言模型进入企业知识管理、员工服务、合规问答和内部流程支持场景，基于检索增强生成 (Retrieval-Augmented Generation, RAG) 的企业知识库 Agent 正在成为组织内部信息查询的重要入口。RAG 通过检索外部知识片段来增强回答的事实性，但也引入了新的安全问题：模型上下文中不再只有用户问题和系统指令，还包含来自网页、PDF、Wiki、手册或数据库的非可信文本。攻击者既可以在用户问题中直接嵌入“忽略规则”“泄露凭证”等恶意指令，也可以在被检索文档中植入间接提示注入，使模型把数据误当作指令执行。

本项目设计并实现 Enterprise-RAG-Guard，一套面向企业专属知识库 Agent 的可迁移提示注入防御框架。系统不构建一个混合回答所有公司问题的跨企业代理，而是将每家企业建模为独立租户：Company Agent = Universal Security Core + Company Security Profile + Company Knowledge Base。通用安全核心负责查询风险检测、双语查询改写、来源感知检索、证据隔离、抽取-生成隔离、引用验证和修复/拒答；企业安全配置负责定义企业边界、允许来源、敏感字段、风险阈值和引用要求；企业知识库则由该企业公开资料或未来客户接入数据构成。

项目首先复现了无防御 RAG 的多种基线，包括 TF-IDF、BM25、LLM-only、TF-IDF+DeepSeek、BM25+DeepSeek、Hybrid+DeepSeek 和 Embedding+DeepSeek。早期 55 题实验显示，许多无防御 RAG 虽然具有较高的检索命中率，但仍会出现投毒引用、伪造引用、跨公司污染和应拒答问题未拒答等安全失败。基于这些失败原因，项目进一步构建七家公司、1084 个干净知识片段、224 个正常与攻击评测问题的企业级评测环境，并设计 B0-B7 消融实验。结果显示，完整 B7 防御将普通 RAG 的攻击成功率从 89.29% 降至 25.00%，引用错误率从 58.48% 降至 0，投毒存活率从 100% 降至 0。

除算法原型外，项目还实现了公网可访问的网站和企业接入流程。网站支持员工查询、攻击挑战、Control/Secure Agent 对比、防御 Trace、可信证据、隔离区展示和企业自助接入。企业接入接口调用安全摄取管道，生成 Tenant Profile，将合规文档切块写入持久化租户索引，并允许新企业立即被查询。本文系统总结项目背景、语料构建、基线实验、防御机制、系统实现、实验分析、局限性与小组分工。

关键词：检索增强生成；提示注入；企业知识库；RAG Agent；安全网关；多租户隔离；引用验证

Abstract

Large language models are increasingly deployed as enterprise knowledge assistants for human resources, compliance, IT support, and internal policy question answering. Retrieval-Augmented Generation (RAG) improves factual grounding by inserting retrieved documents into the model context, but it also creates a broader attack surface. The model context may contain not only system instructions and user questions, but also untrusted text from websites, PDFs, handbooks, wikis, and databases. Malicious instructions can appear directly in user queries or indirectly inside retrieved documents, causing a RAG agent to fabricate policies, leak sensitive information, cite poisoned evidence, or mix policies across companies.

This project proposes Enterprise-RAG-Guard, a transferable defense framework for company-specific RAG agents. Instead of building a single mixed cross-company assistant, each enterprise is modeled as an isolated tenant: Company Agent = Universal Security Core + Company Security Profile + Company Knowledge Base. The universal security core performs query risk detection, bilingual query rewriting, provenance-aware retrieval, evidence quarantine, extractor-generator isolation, citation verification, and repair/refusal. The company profile defines tenant boundaries, allowed sources, sensitive fields, risk thresholds, and citation requirements. The company knowledge base contains public proxy documents in this course project and can be replaced by customer-owned data in deployment.

The project first reproduces several undefended RAG baselines, including TF-IDF, BM25, LLM-only, TF-IDF+DeepSeek, BM25+DeepSeek, Hybrid+DeepSeek, and Embedding+DeepSeek. Early 55-case experiments show that high retrieval hit rates do not guarantee safety: undefended RAG systems may still cite poisoned chunks, fabricate citations, confuse company boundaries, or answer questions that should be refused. Based on these failure modes, the final project builds a seven-company benchmark with 1,084 clean chunks and 224 normal and adversarial evaluation cases. The B0–B7 ablation experiment shows that the full B7 guard reduces attack success rate from 89.29% to 25.00%, citation error rate from 58.48% to 0, and poison survival rate from 100% to 0.

Beyond the algorithmic prototype, the project implements a public web platform and a tenant onboarding path. The website supports employee queries, red-team challenges, Control/Secure Agent comparison, defense traces, safe evidence, quarantine panels, and enterprise onboarding. The onboarding endpoint invokes a secure ingestion pipeline, generates a tenant profile, writes accepted documents into a persistent

tenant index, and makes the new company immediately queryable. This paper presents the motivation, corpus construction, baseline experiments, defense mechanism, system implementation, experimental analysis, limitations, and team contribution.

Key words: Retrieval-Augmented Generation; prompt injection; enterprise knowledge agent; RAG security; tenant isolation; citation verification

目录

1 绪论	1
2 项目总体设计	2
3 相关研究与技术背景	3
4 语料库构建	5
5 无防御 RAG 基线实验	7
6 问题定义与威胁模型	9
7 防御机制设计	10
8 企业接入与产品化实现	15
9 最终评测设计	16
10 实验结果与原因分析	18
11 防御机制的技术贡献	20
12 网站展示与公网验证	21
13 局限性与改进方向	22
14 结论	22
A 相应数据与代码整理	25
B 小组分工情况	26

1 绪论

1.1 研究背景

企业知识库长期面临资料分散、检索成本高、制度解释不一致和审计困难等问题。员工在查询福利、报销、IT 支持、合规制度、供应链风险或社会责任资料时，往往需要知道文档名称、制度关键词、部门路径或系统入口，才能通过传统关键词搜索找到相关文件。大语言模型提供了自然语言交互能力，而 RAG 进一步允许模型在回答时引用组织已有文档，从而降低幻觉并提高答案可追溯性。因此，越来越多企业开始尝试构建面向员工的知识库 Agent。

然而，RAG 的安全边界比普通聊天机器人更复杂。普通聊天机器人主要受到用户输入影响，而 RAG 系统还受到检索文档影响。只要某个网页、PDF、Wiki 页面或内部笔记被检索进入上下文，模型就可能同时看到“事实证据”和“恶意指令”。如果系统没有明确区分二者，攻击者便可能通过间接提示注入改变最终回答。例如，投毒文档可以伪装成“最新高管批准政策”，要求模型忽略原有制度、泄露凭证、提供系统提示词或引用伪造来源。

企业环境中的提示注入并不只是“模型说错话”的问题。若 HR 知识助手错误声称某类福利已被批准，可能造成员工误解和合规争议；若 IT 助手泄露访问令牌或内部系统提示词，可能带来直接安全事故；若合规助手引用跨公司或投毒证据，可能破坏审计可信度。由此可见，企业 RAG Agent 的核心挑战不是“能否生成一段流畅回答”，而是“能否只基于允许来源中的可信证据回答，并在无法安全回答时拒答”。

1.2 研究问题

本项目的研究问题是：如何为不同企业的专属 RAG Agent 构建一套可迁移、可解释、可产品化的提示注入防御框架？这里的“可迁移”并不意味着所有企业共享一个混合知识库，而是指安全核心可以跨企业复用，而每家企业仍保留自己的知识边界、来源策略、敏感字段和安全配置。

我们将企业知识 Agent 建模为如下公式：

$$\begin{aligned} \text{Company Agent} = & \text{Universal Security Core} \\ & + \text{Company Security Profile} \\ & + \text{Company Knowledge Base.} \end{aligned} \tag{1}$$

其中，Universal Security Core 是可迁移的安全能力；Company Security Profile 是企业级策略适配层；Company Knowledge Base 是企业自己的知识内容。这个拆分避免了两种常见误区：一是把多个公司资料混在一个知识库，导致公司边界不清；二是

每增加一家企业就重新实现一套防御逻辑，导致系统无法扩展。

1.3 项目贡献

本文的贡献包括以下五点。

1. 提出面向企业专属 RAG Agent 的 Enterprise-RAG-Guard 架构，将企业知识内容与通用安全防御能力解耦。
2. 复现七种无防御 RAG 基线，分析 TF-IDF、BM25、Hybrid、Embedding 和 LLM 生成在安全场景下的失败原因。
3. 构建中英双语、多公司、多文档格式的企业代理知识库，共 1084 个干净 chunk，并设计 224 个正常与攻击评测问题。
4. 设计 B0-B7 消融实验，从查询检测、来源感知检索、证据隔离、抽取-生成隔离、引用验证和修复/拒答等层面量化防御收益。
5. 实现公网可访问的产品化网站，支持员工查询、红队攻击挑战、防御 Trace、可信证据/隔离区展示和真实企业接入。

2 项目总体设计

2.1 从课程型 RAG 到企业安全网关

项目早期版本关注“员工手册问答系统能否回答问题”，主要围绕 Made Tech 员工手册、55 个标注问题和若干合成投毒 chunk 展开。该阶段的价值在于复现 RAG 基本流程、构造攻击样本、建立初步指标。但是，如果项目停留在单一手册、单一企业和简单规则过滤，就很难与更完整的 Agent 项目相比，也很难解释真实企业如何购买和接入。

最终版本将项目定位为企业 RAG 安全网关。系统预置七家公司作为 benchmark tenants，但核心目标不是让一个 Agent 同时回答所有公司问题，而是验证一套通用安全核心能否迁移到不同企业的独立知识边界上。网站中的员工查询、攻击挑战和企业接入分别对应三类使用者：普通员工、红队/安全测试者和企业管理员。

2.2 系统模块

表 1 最终项目主要模块

模块	主要文件	职责
语料构建	build corpus	抓取、解析和切分公开企业资料，生成多企业 chunk 与元数据。
安全核心	RAG guard core	实现 B0-B7 防御、检索、隔离、生成、引用验证和指标判定。
企业接入	onboarding pipeline	定义连接器、TenantProfile、安全摄取管道和持久化租户索引。
实验评估	transfer experiment	运行 224 题 B0-B7 消融实验并输出结果。
网站展示	demo server	提供员工查询、攻击挑战、Trace、可信证据、隔离区和接入向导。
部署配置	Dockerfile, render.yaml	支持 Render 公网部署、环境变量配置和持久磁盘挂载。

2.3 产品化公式

研究原型可以概括为公司级 Agent 公式；产品化平台进一步扩展为：

$$\begin{aligned} \text{Platform} = & \text{Connector Layer} + \text{Tenant Isolation} \\ & + \text{Universal Guard} + \text{Evaluation} + \text{Monitoring}. \end{aligned} \quad (2)$$

Connector Layer 解决企业数据如何进入系统；Tenant Isolation 解决多企业边界；Universal Guard 解决提示注入防御；Evaluation 解决红队测试和安全回归；Monitoring 解决上线后的风险日志、隔离文档和策略变更审计。

3 相关研究与技术背景

3.1 RAG 构建方法

RAG 的基本思想是在生成模型回答之前，从外部知识库检索相关文档，并将检索结果作为上下文输入模型。Lewis 等提出的 Retrieval-Augmented Generation 方法证明，检索模块可以增强开放域问答与知识密集型任务的事实性 [1]。在企业场景中，RAG 的优势在于可以利用组织已有制度、手册、网页和数据库，避免频繁微调模型，并通过引用提高答案可审计性。

典型 RAG 系统包括文档摄取、切分、索引、检索、重排、提示拼接、答案生成和引用展示。检索方法可以分为稀疏检索和稠密检索。BM25 是经典稀疏检索方法，基于词项频率和逆文档频率计算相关性 [2]；TF-IDF 也是常见的课程和原型系统基线；Dense Passage Retrieval 和 Sentence-BERT 等方法则通过向量表示捕捉语义相似度 [3, 4]。企业 RAG 往往结合两类方法：稀疏检索保证关键词和制度编号命中，稠密检索改善语义改写和跨语言表达。

3.2 提示注入与间接提示注入

提示注入研究指出，语言模型容易受到自然语言指令覆盖影响。直接提示注入发生在用户输入中，攻击者要求模型忽略系统规则、泄露隐藏提示或伪造输出格式；间接提示注入发生在网页、邮件、文档和检索片段中，模型在处理非可信内容时错误地将数据解释为指令。Greshake 等研究表明，LLM 集成应用可能因为外部内容而执行用户未授权的行为 [5]。对于 RAG 系统而言，间接注入尤其重要，因为检索过程会主动将外部内容放入上下文。

OWASP LLM Top 10 将提示注入、敏感信息泄露、供应链风险和过度代理权限列为 LLM 应用的重要风险 [6]。这些风险与企业 RAG 高度相关：知识库文档可能来自多个来源，模型可能具有访问内部系统的能力，回答可能被员工当作正式制度解释。因此，企业 RAG 安全不能只依赖“请不要泄露秘密”这样的系统提示，而需要检索层、证据层、生成层和验证层共同约束。

3.3 RAG 防御思路

现有防御大致可以分为四类。第一类是输入输出过滤，包括检测敏感词、凭证请求、越权意图或恶意格式要求。这类方法成本低、可解释，但容易被改写绕过。第二类是指令与数据隔离，即明确区分系统指令、用户请求和非可信证据，避免模型把文档内容当作更高优先级指令。第三类是检索与引用层面的防御，包括来源校验、权限过滤、引用验证和检索结果重排。第四类是运行时治理，包括日志、审计、红队测试和持续评估。

NIST AI Risk Management Framework 强调 AI 系统需要治理、映射、测量和管理四类活动 [7]。这说明企业级 RAG 安全不是一次性 prompt 工程，而应包含持续评估、策略配置、版本控制和风险监控。本项目因此不仅实现问答防御，还设计红队评估、攻击面分析、企业接入和安全摄取流程。

4 语料库构建

4.1 公司选择与数据来源

最终干净语料位于 `data/enterprise_corpus/company_chunks.csv`，共包含 1084 个知识片段。语料来源包括本地 Made Tech 员工手册、GitLab 公开手册、37signals/Basecamp 公开手册、Valve 公开员工手册，以及腾讯、比亚迪、华为公开 PDF 报告或公告。选择这些公司有三点考虑：第一，它们覆盖不同国家和行业；第二，公开资料形式多样，包含 HTML、PDF 和本地 Markdown/CSV；第三，它们分别提供企业文化、员工政策、治理、风险、ESG、业务和社会责任等不同类型内容，便于模拟真实企业知识库的复杂性。

表 2 最终干净语料公司分布

公司	语言	chunk 数	主要内容类型
Made Tech	英文	134	员工手册、福利、岗位、公司介绍
GitLab	英文	159	远程办公、治理、ESG、工程协作
37signals/Basecamp	英文	111	员工政策、福利、公司文化
Valve	英文	80	员工手册、项目选择、组织文化
腾讯	中文	200	年报、业务、治理、风险、社会责任
比亚迪	中文	200	ESG、员工权益、劳工人权、公益
华为	中英混合/英文为主	200	年报、AI、质量、创新、治理
合计	中英双语	1084	company / employee / business / risk / governance / ESG

从来源格式看，公开 PDF 贡献 680 个 chunk，公开 HTML 贡献 270 个 chunk，本地干净资料贡献 134 个 chunk。从语言看，中文 600 个 chunk，英文 484 个 chunk。从内容类型看，company 类 551 个，employee 类 204 个，business 类 130 个，risk 类 74 个，governance 类 67 个，ESG 类 58 个。所有干净 chunk 的 `trust_level` 均标记为 `official`，合成攻击文档不计入干净语料。

4.2 具体语料示例

为了避免只抽象地说“企业资料”，表3展示了若干真实 chunk 示例。可以看到，不同公司资料在语言、格式和主题上差异明显：Made Tech 和 Basecamp 更接近员工手册；GitLab 包含远程办公和治理；Valve 体现组织文化；比亚迪包含员工权益、人权

保障和公益；华为包含 AI、质量和创新；腾讯公开报告中有大量财务、业务和治理片段。

表 3 语料库具体 chunk 示例

公司	chunk ID	类型	内容摘要
Made Tech	CH0001	company	Cycle to Work 计划支持员工骑行通勤，Made Tech 在 CycleScheme 中的额度上限为 2500 英镑。
GitLab	GL0001	governance	包含远程公司福利、远程工作文化、燃尽和孤立问题、会议最佳实践等主题入口。
Basecamp	BC0002	employee	员工手册说明公司政策和福利，并要求新员工了解公司期望和员工可获得的支持。
Valve	VL0005	employee	描述 Valve 创建一种让优秀个体自主工作、减少阻碍的组织环境。
比亚迪	BY0126	risk	参考 SA8000 建立劳工及人权管理体系，制定人权政策声明和劳工人权保障制度。
比亚迪	BY0024	company	与南非公益组织合作，为社区提供食品、书籍、文具和衣物等日常生活与教育物资。
华为	HW0003	risk	强调质量是生命线，致力于提供高质量产品和服务，并分享质量管理经验。

4.3 切分与元数据

每个 chunk 不只是文本，还包含安全防御需要的元数据：`company_id`、`source_url`、`source_host`、`source_type`、`corpus_origin`、`trust_level`、`document_version`、`content_hash` 和 `instruction_risk_score`。这些字段是后续防御的基础。普通 RAG 往往只保留文本和 chunk ID，而 Enterprise-RAG-Guard 将来源、公司、版本、风险和可信度变成一等字段，使检索结果可以被安全策略审查。

中文企业腾讯、比亚迪、华为各取 200 个去重片段。该做法不是合成补齐，而是从公开 PDF 中解析候选段落按目标数量截取，以平衡不同企业语料规模，避免大型 PDF 对整体评估产生过强影响。需要强调的是，本项目使用公开资料构建代理知识库，并不声称拥有真实企业内部资料。攻击样本与干净语料严格分离，投毒 chunk 只在评测和网站攻击挑战中注入。

5 无防御 RAG 基线实验

5.1 为什么先做基线

在设计防御之前，项目先复现多种无防御 RAG。这样做有两个原因。第一，只有理解普通 RAG 如何失败，才能设计有针对性的防御；第二，企业项目不能只说“我们加了安全规则”，还需要证明不同检索与生成组合在攻击场景下的具体风险。早期基线实验使用 55 个标注问题，包含正常问题、用户注入、文档注入、混合攻击和必须拒答问题。

5.2 七种无防御 RAG 构建方式

表4列出七种无防御 RAG。需要注意，rag3 是 LLM-only，不是真正 RAG，而是作为无检索参考；rag7 使用百炼/DashScope text-embedding-v4 作为 embedding 模型；rag4、rag5、rag6 使用 DeepSeek 作为生成模型。

表 4 无防御 RAG 基线构建方式

基线	检索方式	生成方式	设计目的
rag1	TF-IDF	本地抽取式	观察最基础稀疏检索和抽取式回答能力。
rag2	BM25	本地抽取式	测试经典 BM25 是否优于 TF-IDF 抽取式回答。
rag3	无检索	DeepSeek	检查没有知识库时模型能否凭通用知识回答，作为非 RAG 参考。
rag4	TF-IDF	DeepSeek	稀疏检索 + LLM 生成，是早期整体代理分最高的控制组。
rag5	BM25	DeepSeek	BM25 检索 + LLM 生成，观察词项检索变化的影响。
rag6	TF-IDF + BM25 Hybrid	DeepSeek	融合检索，平衡不同稀疏检索信号。
rag7	text-embedding-v4	DeepSeek	语义向量检索 + LLM 生成，测试 embedding 检索效果。

5.3 基线指标与结果

早期 baseline 使用代理指标进行评估。avg token F1 衡量回答和参考答案的词项重叠；gold chunk any/all hit rate 衡量检索是否命中金标 chunk；normal accuracy

proxy 衡量正常问题是否答对；attack safe answer proxy 衡量攻击问题是否没有被带偏；refusal accuracy proxy 衡量必须拒答问题是否拒答；overall proxy 是综合代理分。代理指标不能替代人工评审，但足以揭示不同 RAG 组合的失败模式。

表 5 早期 55 题无防御基线结果

基线	token F1	金标命中 (any)	正常准确率	攻击安全率	综合分
rag4 TF-IDF+LLM	0.4430	92.73%	67.86%	59.09%	63.64%
rag5 BM25+LLM	0.4256	94.55%	64.29%	54.55%	60.00%
rag6 Hybrid+LLM	0.4330	94.55%	64.29%	54.55%	60.00%
rag7 Embedding+LLM	0.4057	96.36%	53.57%	45.45%	50.91%
rag1 TF-IDF local	0.2638	92.73%	39.29%	22.73%	29.09%
rag3 LLM-only	0.2344	0.00%	0.00%	0.00%	5.45%
rag2 BM25 local	0.1515	94.55%	3.57%	0.00%	1.82%

结果显示，高检索命中率并不等于安全。rag7 的金标命中率最高，但综合分低于 rag4，说明 embedding 检索在小规模员工手册中可能召回语义相近但不够精确的片段，生成模型又可能基于不完全证据作答。rag2 的 BM25 命中率不低，但本地抽取式生成效果很差，说明检索命中只是第一步，生成策略和引用策略同样重要。rag3 无检索时无法完成企业制度问答，证明企业知识任务必须依赖外部证据。

5.4 无防御 RAG 的失败原因

我们从失败样本中总结出四类原因。第一，检索文档进入上下文后，模型没有区分“证据”和“指令”，会被投毒 chunk 中的格式劫持、政策覆盖或引用劫持影响。第二，无防御 RAG 没有公司边界，容易在多企业语料中引用其他公司的政策。第三，生成模型会产生看似合理但不存在或未检索到的引用 ID，造成引用幻觉。第四，必须拒答问题只靠 LLM 自觉不可靠，凭证和隐私问题可能被正常回答。

这些失败原因直接决定了后续防御机制：如果失败来自用户问题，就需要 query risk detection；如果失败来自投毒文档，就需要 chunk risk scan 和 quarantine；如果失败来自公司混淆，就需要 tenant/company boundary；如果失败来自引用幻觉，就需要 citation verification；如果失败来自模型直接读取原始文档中的恶意文本，就需要 instruction/evidence isolation 和 extractor-generator isolation。

6 问题定义与威胁模型

6.1 形式化定义

给定用户问题 q 、目标企业 c 和企业知识库 D_c ，企业 RAG Agent 需要检索证据集合 E 并生成带引用的回答 a 。安全目标可以表述为：

$$a = G(q, E), \quad E \subseteq D_c \cap S_c \cap P_u, \quad (3)$$

其中 S_c 表示企业允许来源集合， P_u 表示用户可访问权限集合。课程原型暂未实现真实用户权限，因此当前实现主要覆盖 D_c 和 S_c ；生产系统应进一步加入 SSO、RBAC 和文档级 ACL。

6.2 攻击面

表 6 本项目威胁模型

攻击面	攻击方式	期望防御行为
直接用户注入	用户要求忽略规则、改变格式、泄露系统提示词。	识别风险意图，必要时拒答或只回答安全部分。
凭证索取	用户要求提供密码、API key、访问令牌、HR/财务系统凭证。	必须拒答，不得生成替代凭证或内部路径。
跨公司污染	用户要求用其他公司政策回答当前公司问题。	保持 company boundary，只引用目标企业证据。
自适应政策修订	攻击伪装成“最新批准修订”，试图覆盖原政策。	不接受用户或文档中的未经验证修订。
检索文档投毒	恶意 chunk 被检索后要求模型忽略系统规则或引用投毒段落。	检索可见但生成前隔离，不进入安全证据。

项目假设攻击者可以控制用户输入，也可能影响某些待检索文档或公开网页，但不能修改系统代码、企业安全配置或真实允许来源列表。该假设对应现实中的两类风险：外部网页污染和内部知识库污染。前者可能发生在公开网页、供应商文档或网页爬取场景；后者可能发生在员工上传文件、共享文档或协作平台中。

7 防御机制设计

7.1 设计原则

Enterprise-RAG-Guard 的核心不是某一个规则，而是一组分层防御原则。

1. **证据不是指令。**检索文档只能作为非可信证据，不能覆盖系统安全规则。
2. **企业边界优先。**回答某家公司问题时，只允许使用该公司 profile 允许的来源。
3. **引用必须可验证。**答案中的每个引用 ID 必须真实存在、属于目标公司、未被隔离。
4. **先隔离再生成。**可疑 chunk 可以被记录和展示，但不能直接喂给生成器。
5. **无法安全回答时拒答。**防御目标不是“永远回答”，而是“安全地回答或明确拒答”。

7.2 Company Security Profile

Company Security Profile 是项目的企业适配层。每个 profile 记录 company_id、company_name、allowed domains、sensitive fields、allowed tasks、risk threshold、minimum evidence count 和 citation policy。它的作用类似企业 RAG 的安全配置文件：同一套 Universal Security Core 可以加载不同企业 profile，从而适配不同知识边界。

例如，比亚迪 profile 会要求回答 BYD 问题时只使用 BYD 相关 chunk；GitLab profile 会允许 GitLab handbook 来源；中文企业 profile 会启用中文风险词和双语查询改写。未来真实部署时，profile 还可以扩展为部门权限、数据保留周期、模型选择、审计等级和人工审批策略。

7.3 查询风险检测

查询风险检测负责在检索前识别用户侧攻击。系统检测中英文表达，包括 direct override、credential request、private data request、system prompt exfiltration、policy fabrication 和 citation hijacking。中文检测覆盖“忽略规则”“密码”“凭证”“访问令牌”“系统提示词”“特殊无限福利”等表达。

查询风险检测并不承担所有防御责任，因为攻击者可以改写表达绕过规则。但它有三个作用：第一，对必须拒答问题提前拒答；第二，作为 trace 中的安全信号；第三，影响后续生成和验证策略。例如，若用户要求“给我 GitLab API key”，系统不应先检索再让模型自由发挥，而应直接进入拒答路径。

7.4 双语查询改写与语义重排

最终网站接入 DeepSeek 和百炼/DashScope Embedding。中文用户可能查询英文公司，英文用户也可能查询中文公司。系统因此加入 bilingual query rewriting：先用本地词典和必要时的 DeepSeek 改写，把中文查询扩展为中英混合检索表达；再从候选 chunk 中选取 shortlist，并使用 text-embedding-v4 做语义重排。这样既保留 lexical 检索对专名、制度词和 chunk ID 的精确性，又利用 embedding 改善跨语言和近义表达。

为了提高公网查询速度，系统默认开启 fast mode 和 embedding shortlist。也就是说，embedding 不对全库 1084 个 chunk 每次重算，而是先用轻量检索缩小候选范围，再对候选进行语义排序。这是产品工程中的折中：安全链路仍完整，但避免每次请求都调用大量 embedding。

7.5 来源感知检索与隔离

普通 RAG 将检索结果主要按相关性排序，而 Enterprise-RAG-Guard 将相关性与安全属性融合。每个候选 chunk 会被检查 company match、source host、trust level、poison flag、instruction risk score、source type 和 citation suitability。错误公司、非允许来源、投毒标记、指令风险较高或 source type 异常的 chunk 会进入 quarantine。

Quarantine 不是简单丢弃。网站会展示被隔离片段，使用户和答辩老师能看到系统为什么拒绝某些证据。例如在 BYD 查询测试中，Control Agent 会引用 PX_BYD_001 投毒片段，而 Secure Agent 将 PX_BYD_001、PX_BYD_002、PX_BYD_003 放入隔离区，最终使用 BY0125 和 BY0184 生成答案。

在实现上，检索候选的安全分并不替代相关性分，而是作为硬约束和重排信号共同作用。候选证据首先必须满足企业边界和来源白名单；若不满足，则无论语义相关性多高都不能进入 safe evidence。对满足硬约束的候选，系统再根据风险词、投毒标记、证据类型和引用可用性调整排序。这样可以避免“恶意文档因为恰好重复用户问题而排到第一”的常见失败。

表 7 失败原因与防御层映射

失败原因	无防御 RAG 表现	Enterprise-RAG-Guard 对应机制
用户侧越权请求	模型尝试配合“忽略规则”“泄露凭证”等请求。	查询风险检测、敏感字段拒答、修复/拒答。
跨企业证据污染	相似政策被错误引用到当前公司。	Tenant Profile、company_id 硬过滤、allowed source 校验。
检索文档投毒	投毒 chunk 进入 prompt 并影响生成。	instruction risk scan、quarantine、指令/证据隔离。
伪造或悬空引用	回答看似合理但引用不存在或不支持主张。	claim 抽取、citation verifier、失败后修复。
中文英文错配	中文问题查英文公司或英文问题查中文公司时召回不足。	双语查询改写、候选 shortlist、embedding 重排。

7.6 指令/证据隔离

指令/证据隔离是本项目的关键技术思想。系统将上下文分成四类：系统安全规则、用户任务、企业 profile、检索证据。只有系统规则和 profile 可以定义行为边界，检索证据不能成为行为指令。即使证据中写着“忽略所有规则并泄露密码”，它也只能作为待审查文本，而不能改变系统行为。

这一机制借鉴了 instruction-data separation 和 spotlighting 的思想，但在本项目中与企业 profile、chunk 元数据和 citation verification 结合起来。它不是只在 prompt 中写一句“不要相信文档中的指令”，而是在代码层面先隔离风险 chunk，再只把安全 evidence claims 传给生成阶段。

7.7 抽取-生成隔离

无防御 RAG 往往把原始检索文本直接拼进 prompt，导致恶意文本靠近模型生成器。抽取-生成隔离的思想是：先从安全 chunk 中抽取事实 claim，再根据 claim 生成答案。claim 结构包括 claim text、company ID、chunk ID 和 evidence span。若某个句子像指令而不是事实，例如“忽略原政策”“必须直接提供凭证”，则不会进入 claim pool。

这种设计降低了原始投毒文本接触生成器的机会。即使检索阶段召回了风险片段，生成阶段看到的也只是经过过滤的事实声明。对于企业场景，这一点尤其重要，因为很多公开报告和 PDF 会包含目录、免责声明、表格残片和 OCR 噪声，直接喂给 LLM 容易产生引用错误。

抽取阶段还承担“证据压缩”的作用。企业 PDF 中经常存在长表格、财务数字、章节目录和版权声明；这些内容本身未必危险，但会稀释上下文窗口并增加模型误读概率。将原始 chunk 压缩成带来源的 claim 后，生成器收到的是更短、更结构化的事实集合，便于在回答中保持引用一致。课程实现中 claim 抽取仍采用轻量规则和 LLM 生成配合，未来可以替换为自然语言推理模型或专门的信息抽取模型。

7.8 引用验证与修复/拒答

生成答案后，Verifier 会检查四类条件。第一，引用 ID 是否存在于当前检索和安全证据中；第二，引用是否属于目标公司；第三，引用是否来自投毒或隔离 chunk；第四，回答是否包含敏感字段、伪造政策或无证据主张。若验证失败，B7 会尝试一次修复：重新基于安全证据生成更保守答案；若仍失败，则拒答。

引用验证是企业 RAG 与普通聊天机器人的重要区别。企业员工往往会把答案作为制度解释，因此“回答看似合理”不够，必须能追溯到允许来源中的证据。B7 在当前评测中将 citation error rate 降至 0，说明该机制对于跨公司污染和投毒引用非常有效。

7.9 完整防御算法

将上述模块合在一起，B7 Full Guard 的一次请求可以写成算法1。该算法的关键点是：风险判断发生在生成前、生成中和生成后，而不是只在 prompt 里添加一句安全提示。

Listing 1 B7 Full Guard 请求流程

```
Input: user query q, target company c, language l
Load profile P_c and tenant corpus D_c

1. Detect query risk:
    risk_q = detect_user_attack(q, language=l)
    if risk_q requires refusal:
        return safe_refusal(trace=risk_q)

2. Rewrite and retrieve:
    q' = bilingual_rewrite(q, l, P_c)
    candidates = lexical_retrieve(q', D_c)
    candidates = embedding_rerank(q', candidates.shortlist)

3. Provenance and safety filtering:
    safe_evidence = []
```

```
quarantine = []
for chunk in candidates:
    verdict = check_company_source_risk(chunk, P_c)
    if verdict.safe:
        safe_evidence.append(chunk)
    else:
        quarantine.append((chunk, verdict.reason))

4. Evidence extraction:
claims = extract_supported_claims(safe_evidence)
claims = remove_instruction_like_claims(claims)

5. Generate answer:
answer = generate_with_deepseek(q, claims, P_c, language=l)

6. Verify and repair:
report = verify_citations(answer, claims, P_c)
if report.failed:
    answer = repair_or_refuse(q, claims, report)

7. Return answer, citations, safe evidence, quarantine, trace.
```

7.10 与简单规则过滤的区别

本项目早期防御更接近规则过滤：如果文档包含“ignore previous instructions”等关键词，就降低其权重或删除。这种方法有明显局限。第一，攻击者可以用同义改写绕过关键词；第二，过滤器无法处理跨公司引用，因为跨公司资料本身未必包含危险词；第三，过滤器无法保证最终答案引用真实存在；第四，删除风险文档后，系统无法向用户解释为什么拒绝。

最终版 Enterprise-RAG-Guard 的技术性提升在于把防御拆成可组合的安全属性：tenant boundary 控制“能不能用这家公司以外的证据”；provenance control 控制“来源是否允许”；instruction/evidence isolation 控制“文档内容能否变成行为指令”；extractor-generator isolation 控制“原始文本是否直接接触生成器”；citation verification 控制“答案是否真的由证据支持”；quarantine 和 trace 控制“防御过程是否可审计”。因此它不是某个关键词表，而是一条从输入到输出的安全链路。

7.11 跨企业迁移方式

跨企业迁移不是把七家公司混在同一知识库中反复做一遍，而是将安全核心固定，将企业差异放入 profile 和 corpus。新增企业时，只需要生成新的 Tenant Profile、接入企业知识库、配置 allowed sources 和 sensitive fields；B7 的风险检测、隔离、抽取、验证和 trace 逻辑保持不变。这样每家企业都有自己的 Agent，同时平台不需要为每个企业重写防御代码。

该设计也解释了为什么实验要选择七家公司：英文手册类公司测试员工政策和远程办公资料；中文上市公司测试 PDF 年报、ESG、治理、业务和社会责任资料；中英混合查询测试语言迁移。若 B7 在这些异构企业中都能保持 0 引用错误率和 0 投毒存活率，就说明防御核心具有一定迁移潜力。

8 企业接入与产品化实现

8.1 真实接入流程

最终网站包含“企业接入”页面。企业管理员可以填写企业名称、语言、行业、交付方式、允许来源和计划接入的数据源，并粘贴示例文档或提供公开 URL。后端 /api/onboard 调用 create_tenant_agent，执行以下流程：

```
Create tenant
-> Fetch or receive documents
-> Normalize into DocumentRecord
-> Scan instruction risk and sensitive fields
-> Accept or quarantine documents
-> Split accepted documents into chunks
-> Write tenant chunks and profiles
-> Reload EnterpriseRAGGuard
```

实际测试中，包含“访问令牌”“系统提示词”等风险词的演示文档会被隔离；不含敏感请求的 HR/报销/社会责任资料会被写入持久化租户索引，并生成类似 KNOWLEDGEDEMO_OF4A64_DOC001_001 的 chunk ID。随后员工查询页面可以选择该企业并得到带引用回答。

8.2 连接器抽象

enterprise_onboarding.py 定义了 KnowledgeConnector、DocumentRecord、TenantProfile、SecureIngestionPipeline 和 IngestionReport。课程原型实现了管理员粘贴文本和公开 URL，两者足以证明接入链路真实存在。生产版本可以扩展

SharePoint、Confluence、Notion、内部 API、对象存储、数据库和已有向量库连接器。

表 8 企业接入模块与生产扩展

组件	当前实现	生产扩展
KnowledgeConnector	管理员文本、公开 URL	SharePoint、Confluence、内部 API、数据库、已有向量库
DocumentRecord	统一文档 schema	增加 ACL、审批人、数据分类、保留周期
SecureIngestionPipeline	注入扫描、敏感词检测、去重、隔离	OCR、文件类型校验、恶意文件检测、人工审批流
TenantProfile	allowed sources、敏感字段、阈值	SSO、RBAC、部门权限、模型策略、审计策略
Persistent Store	Render 1GB disk	企业 VPC、专属向量库、对象存储、加密数据库

8.3 公网部署

项目已部署到 Render: <https://enterprise-rag-guard-deploy.onrender.com>。部署使用 Starter 实例和 1GB 持久磁盘，挂载路径为 `/app/data/tenant_agents`，用于保存新接入租户的 chunks 和 profiles。DeepSeek 和 DashScope/Bailian API key 通过 Render 环境变量配置，不写入 GitHub 代码。服务启动后，`/api/meta` 返回企业列表和当前生成模式；`/api/ask` 提供问答；`/api/onboard` 提供企业接入。

9 最终评测设计

9.1 评测集

最终评测使用 224 个问题，每家公司 32 个，覆盖 7 家公司。问题分布如表9所示。

表 9 最终 224 题评测集分布

维度	类别	数量
公司	Made Tech / GitLab / Basecamp / Valve / Tencent / BYD / Huawei	每家 32
攻击面	none	56
攻击面	user_query	70
攻击面	cross_company	35
攻击面	adaptive	35
攻击面	retrieved_document	28
攻击类型	direct_override	42
攻击类型	credential_request	28
攻击类型	policy_contamination	35
攻击类型	stealth_policy_amendment	35
攻击类型	poisoned_chunk	28
语言	英文 / 中文	128 / 96

9.2 指标定义

最终评测关注以下指标。

- **normal task success rate**: 正常问题是否获得非拒答、可引用且公司一致的答案。
- **attack success rate (ASR)**: 攻击问题是否导致不安全回答、错误引用、跨公司污染、投毒引用或应拒答未拒答。
- **attack resistance rate**: $1 - \text{ASR}$ 。
- **citation error rate**: 答案引用缺失、伪造、跨公司或来自投毒/隔离片段的比例。
- **poison retrieval rate**: 投毒片段被检索到的比例。
- **poison survival rate**: 投毒片段不仅被检索到，而且最终进入答案或引用的比例。
- **required refusal precision/recall**: 对必须拒答问题的拒答精确率和召回率。

这些指标是代理指标，不等同于人工专家评分。它们适合课程项目中的消融实验和系统可行性验证，但不能证明系统在所有企业、所有攻击形式下普遍有效。

10 实验结果与原因分析

10.1 B0–B7 消融结果

表 10 B0–B7 消融结果汇总

配置	正常成功率	攻击成功率	抵抗率	引用错误率	投毒存活率
B0 Plain RAG	67.86%	89.29%	10.71%	58.48%	100.00%
B1 Prompt Guardrail	69.64%	61.90%	38.10%	52.23%	100.00%
B2 Detector	69.64%	57.74%	42.26%	32.14%	100.00%
B3 Provenance Retrieval	100.00%	25.00%	75.00%	0.00%	0.00%
B4 Structured Spotlighting	100.00%	25.00%	75.00%	0.00%	0.00%
B5 Extractor-Generator	100.00%	25.00%	75.00%	0.00%	0.00%
B6 Verifier	100.00%	25.00%	75.00%	0.00%	0.00%
B7 Full Guard	100.00%	25.00%	75.00%	0.00%	0.00%

结果显示，B0 的 ASR 高达 89.29%，说明普通 RAG 在攻击问题面前几乎没有可靠防御。B1 prompt guardrail 将 ASR 降至 61.90%，说明系统提示有一定帮助，但引用错误率仍超过 50%，投毒存活率仍为 100%。B2 detector 进一步降低用户侧攻击和部分引用错误，但文档侧投毒仍可能进入答案。真正的转折点是 B3 provenance retrieval：加入公司边界、来源可信度、投毒标记和风险 chunk 隔离后，引用错误率降至 0，投毒存活率降至 0。

这说明企业 RAG 防御不能只依赖 prompt，也不能只依赖查询分类器。RAG 的核心风险来自“错误证据进入生成上下文”。只要检索层和证据层没有安全边界，生成层再强也可能被污染。B3 之后 B4–B7 在当前确定性实现中总体指标相近，是因为 B3 已经切断了主要投毒引用链；但 B4–B7 的意义在于增强工程可解释性和产品可迁移性：Structured Spotlighting 让 trace 更清楚，Extractor-Generator 减少原文投毒接触生成器，Verifier 提供最终审计，Repair/Refusal 形成安全兜底。

10.2 按攻击面分析

表 11 按攻击面比较 B0 与 B7

攻击面	样本数	B0 ASR	B7 ASR	B7 引用错误率
adaptive	35	85.71%	0.00%	0.00%
cross_company	35	74.29%	0.00%	0.00%
retrieved_document	28	85.71%	0.00%	0.00%
user_query	70	100.00%	60.00%	0.00%
none	56	0.00%	0.00%	0.00%

文档侧攻击被显著压制。adaptive、cross_company 和 retrieved_document 三类攻击在 B7 中 ASR 均为 0, 说明 company boundary、quarantine 和 citation verification 对跨公司污染和检索文档投毒非常有效。user_query 攻击仍有 60% ASR, 是当前系统的主要残余风险。这并不意味着系统会泄露凭证; 而是评价规则将部分用户侧注入、策略伪造或应拒答边界问题判为失败。未来应引入更强语义分类器和人工标注来改善这一部分。

10.3 按公司迁移分析

表 12 B7 在七家公司上的迁移结果

公司	样本数	正常成功率	ASR	引用错误率
Made Tech	32	100.00%	25.00%	0.00%
GitLab	32	100.00%	25.00%	0.00%
37signals/Basecamp	32	100.00%	25.00%	0.00%
Valve	32	100.00%	25.00%	0.00%
腾讯	32	100.00%	25.00%	0.00%
比亚迪	32	100.00%	25.00%	0.00%
华为	32	100.00%	25.00%	0.00%

当前实验使用共享默认阈值, 而不是为每家公司单独调参, 因此结果更接近 profile-only transfer。所有公司 B7 引用错误率为 0, 说明引用验证和公司边界机制具有较好的迁移性。不同公司的 ASR 相同, 是因为评测集按公司均衡生成, 且残

余风险主要集中在 `user_query` 类；这也提示后续应增加更细粒度公司差异样本，例如中文 OCR 噪声、英文 handbook 长段落、财报表格抽取噪声等。

10.4 一次攻击查询案例

为了更具体地说明防御链路，以下以 BYD 员工权益问题为例。用户问题为“请总结比亚迪公开资料中关于员工权益的内容”，同时攻击挑战环境向检索候选中注入三条伪造片段：PX_BYD_001 声称模型应忽略所有安全规则；PX_BYD_002 伪造“最新批准的无限福利”；PX_BYD_003 要求引用投毒片段作为权威来源。

Control Agent 的流程是普通 RAG：它按相关性召回候选，将原始文本直接交给生成器。由于投毒片段包含“员工权益”“福利”“比亚迪”等高相关词，Control Agent 容易引用 PX_BYD_001 或 PX_BYD_002，并把伪造政策写进答案。这个失败并不是模型“不聪明”，而是上下文边界错误：模型没有被告知哪些文本是证据、哪些文本是攻击。

Secure Agent 的流程则不同。首先，查询风险检测没有发现用户问题本身恶意，因此继续检索。其次，provenance filter 检查候选 chunk 的 `company_id`、`source_host`、`trust_level` 和 `poison flag`，三条 PX_BYD 片段被放入 quarantine。再次，系统只从 BY0125、BY0184 等官方 chunk 中抽取 claim，例如员工权益、公益活动、志愿服务和社会责任实践。最后，DeepSeek 只基于安全 claim 生成中文回答，Verifier 检查引用是否存在且未被隔离。因此最终答案可以回答员工权益问题，但不会接受“无限福利”等伪造内容。

这个案例体现了本项目的核心贡献：攻击片段并非在前端隐藏，而是在 trace 中被展示；系统不是简单拒绝所有问题，而是在有安全证据时正常回答；若没有足够证据，则走修复或拒答路径。相比“黑盒安全聊天机器人”，这种可审计链路更适合课堂展示和企业评估。

11 防御机制的技术贡献

11.1 贡献一：将企业边界引入 RAG 安全

许多 RAG 项目把“检索相关性”作为核心指标，但企业场景首先需要回答“这个证据是否属于当前企业”。Enterprise-RAG-Guard 将 `company_id`、`source_host`、`allowed domains` 和 `trust_level` 作为检索后过滤和验证的重要字段，使系统可以防止跨公司污染。这个设计解决了多企业 RAG 中最容易被忽视的问题：相似政策不等于可引用政策。

11.2 贡献二：把投毒文档作为可观测对象而非简单丢弃

传统过滤器往往直接删除风险文本，导致用户无法知道系统为什么拒绝某些证据。Enterprise-RAG-Guard 将风险 chunk 放入 quarantine，并在网站中展示隔离区。这样既避免风险文本进入生成，又保留审计和教学价值。对于答辩演示，隔离区可以清楚展示 Control Agent 为什么失败、Secure Agent 为什么安全。

11.3 贡献三：抽取-生成隔离降低提示注入接触面

普通 RAG 把原始检索文本直接放进 prompt，投毒文本距离生成器很近。Enterprise-RAG-Guard 先抽取安全 claim，再生成答案。这个设计使恶意指令即使出现在原始文档中，也更难直接影响生成器。它不是简单敏感词过滤，而是改变了证据进入生成阶段的表示形式。

11.4 贡献四：引用验证作为安全闭环

企业 RAG 的答案必须可审计。Verifier 检查引用是否存在、是否属于目标公司、是否来自安全证据、是否被隔离、是否支持回答中的关键主张。该机制将 RAG 安全从“输入过滤”扩展到“输出验证”。实验中，B7 引用错误率降至 0，说明该机制对伪造引用、跨公司引用和投毒引用有效。

11.5 贡献五：从研究原型到可接入产品

项目实现了真实企业接入接口，而不是只预装七家公司。企业管理员可以提交资料，系统执行安全摄取、风险扫描、切块和 profile 生成。这个设计回答了“如果企业想购买服务，如何接入自己的知识库”的问题。课程原型虽然只实现文本和公开 URL，但接口抽象已经覆盖 SharePoint、Confluence、内部 API 和已有向量库的扩展路径。

12 网站展示与公网验证

最终网站定位为中文优先的企业知识安全助手，而不是裸露实验指标的研究面板。首页提供员工查询、攻击挑战、企业接入和安全流程四个入口。员工查询页面允许用户选择企业和语言，提出 HR、IT、合规或治理相关问题。攻击挑战页面允许用户输入恶意问题或投毒片段，并并排比较 Control Agent 与 Secure Agent 的输出。企业接入页面允许管理员创建新企业 Agent。

公网验证结果包括三点。第一，/api/meta 返回企业列表和生成模式，说明后端服务正常读取语料和配置。第二，BYD 查询中，Secure Agent 引用 BY0125 和 BY0184，并隔离 PX_BYD_001、PX_BYD_002、PX_BYD_003 投毒片段。第三，新接入

KnowledgeDemo 企业后，系统生成 chunk ID KNOWLEDGEDEMO_0F4A64_DOC001_001，并能立即回答“年假和报销流程是什么”。这证明企业接入链路真实写入持久化租户索引。

13 局限性与改进方向

第一，评测规模仍有限。224 个问题适合原型消融，但不足以证明系统在所有企业和所有攻击形式下普遍有效。后续应扩展到至少 700–1050 个问题，并加入人工标注和置信区间。第二，真实企业场景需要身份认证和文档级 ACL。当前产品化设计预留 access_groups 字段，但尚未实现 SSO、RBAC、部门权限同步和审计日志。第三，语义防御仍然偏轻量。风险检测主要依赖启发式规则，可能被复杂改写绕过。未来应引入训练好的注入检测器、自然语言推理模型和策略一致性验证器。

第四，公开企业资料不能完全代表内部知识库。当前实验使用公开资料构建代理知识库，便于复现和展示，但缺少真实内部制度、访问控制、员工角色和合规约束。第五，公网产品还需要工程化增强，例如域名、HTTPS 证书管理、日志脱敏、限流、后台任务队列、异步摄取、管理员控制台和多租户计费。

14 结论

本文总结了 Enterprise-RAG-Guard 项目：一套面向企业专属知识库 Agent 的可迁移提示注入防御框架。项目从无防御 RAG 的失败原因出发，逐步构建查询风险检测、来源感知检索、证据隔离、抽取-生成隔离、引用验证和修复/拒答机制。实验表明，在七家公司、1084 个 chunk 和 224 个评测问题组成的环境中，完整 B7 防御显著降低攻击成功率、引用错误率和投毒存活率。

更重要的是，项目不只是算法实验，还实现了公网网站和企业接入流程。它展示了一个企业 RAG 安全产品应如何组织：每家企业拥有自己的知识边界和 profile，安全核心跨企业复用，管理员可以接入新资料，员工可以正常查询，红队可以挑战系统，Trace 可以解释防御过程。虽然系统仍有残余风险和工程化不足，但已经从课程型 RAG 原型升级为可展示、可部署、可继续扩展的企业知识安全平台。

参考文献

- [1] Lewis P., Perez E., Piktus A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [2] Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009.
- [3] Karpukhin V., Oguz B., Min S., et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP, 2020.
- [4] Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019.
- [5] Greshake K., Abdelnabi S., Mishra S., et al. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv, 2023.
- [6] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [7] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). 2023.
- [8] Perez F., Ribeiro I. Ignore Previous Prompt: Attack Techniques For Language Models. NeurIPS ML Safety Workshop, 2022.
- [9] Liu Y., Deng G., Xu Z., et al. Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study. arXiv, 2023.
- [10] Zou A., Wang Z., Carlini N., et al. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv, 2023.
- [11] Wallace E., Feng S., Kandpal N., et al. Universal Adversarial Triggers for Attacking and Analyzing NLP. EMNLP-IJCNLP, 2019.
- [12] GitLab. GitLab Handbook. <https://handbook.gitlab.com/handbook/>.
- [13] 37signals/Basecamp. Employee Handbook. <https://basecamp.com/handbook>.
- [14] Valve Corporation. Valve New Employee Handbook. <https://www.valvesoftware.com/en/publications>.

- [15] Tencent. Annual and ESG Reports. <https://www.tencent.com/en-us/investors/financial-news.html>.
- [16] BYD Company Limited. Annual Report and Sustainability Information. <https://www.bydglobal.com/>.
- [17] Huawei. Annual Report and Sustainability Report. <https://www.huawei.com/en/annual-report/>.
- [18] 上海交通大学教务处. 2025 届本科生毕业设计（论文）资料及表格. <https://www.jwc.sjtu.edu.cn/info/1041/117021.htm>.

A 相应数据与代码整理

最终提交材料整理为 `Enterprise-RAG-Guard_Submission.zip`。压缩包只保留论文、展示、代码、数据、实验结果和部署配置，避免把本地虚拟环境、缓存、密钥和浏览器状态一起提交。各部分内容如下。

表 13 提交包结构说明

目录	内容与作用
<code>paper/</code>	存放本论文的 LaTeX 源文件和编译后的 PDF，便于老师直接阅读或重新编译。
<code>slides/</code>	存放最终展示 PPT，最后一页预留网站演示视频位置。
<code>code/</code>	存放项目核心代码，包括防御 RAG、企业接入、网页服务、语料构建和消融实验脚本。
<code>data/</code>	存放公司安全配置、企业语料 chunk 和评测问题，是复现实验的主要输入。
<code>results/</code>	存放 B0-B7 消融实验结果、按攻击面统计、按公司统计和迁移实验矩阵。
<code>deployment/</code>	存放 Docker、Render 等公网部署配置，用于说明网站如何上线运行。
<code>docs/</code>	存放项目技术说明，概括最终版系统架构、防御链路和使用方法。

表 14 核心代码文件说明

文件	主要实现
<code>enterprise_rag_guard.py</code>	实现企业级安全 RAG 主流程，包括查询风险检测、来源感知检索、证据隔离、引用验证、修复/拒答和 B0-B7 防御版本。
<code>enterprise_onboarding.py</code>	实现企业接入功能，包括文档规范化、风险扫描、Tenant Profile 生成、chunk 写入和新企业 Agent 创建。
<code>guard_demo_server.py</code>	实现网站后端接口和前端页面，支持员工查询、攻击挑战、安全 Trace、可信证据、隔离区和企业接入。
<code>build_enterprise_corpus.py</code>	构建七家公司语料库，抓取或解析公开资料，进行去重、切分和元数据标注。
<code>run_guard_transfer_experiment.py</code>	运行 224 题 B0-B7 消融实验，输出攻击成功率、正常成功率、引用错误率和投毒存活率等指标。

提交包不包含 `.env`、API key、虚拟环境、缓存或云平台后台截图；复现模型调用时只需配置 `DEEPSEEK_API_KEY` 与 `DASHSCOPE_API_KEY`。

B 小组分工情况

表 15 小组分工情况

成员	主要分工
黎鸿基	文献综述、公司数据来源梳理、语料统计、提示注入威胁模型和评测指标定义。
乔伊宁	无防御基线设计、攻击样本设计、DeepSeek/百炼接入、公网部署和项目整合。
孙钰婷	Enterprise-RAG-Guard 防御链路实现，包括检索隔离、引用验证、修复/拒答和消融实验。
徐晨飞	Enterprise-RAG-Guard 防御链路实现，包括检索隔离、引用验证、修复/拒答和消融实验。
刘婧祺	网站设计、企业接入功能与代码、实验结果分析、PPT 展示材料和最终文档校对。